

Northwestern



Using Quest Effectively at CIERA

REU Program Workshop

Michail Damoulakis

Computational Specialist, CIERA

From NUIT Basics to CIERA Research Workflows

NUIT Quest Workshop

General Quest basics

- Logging in to Quest
- Command-line navigation
- Submitting a simple Slurm job
 - Quest OnDemand
- General docs and policies

CIERA REU Workshop

Using Quest well for CIERA research

- CIERA allocations and partitions
- Choosing CPUs, memory, GPUs, walltime
- Writing readable batch scripts
 - Job arrays for repeated runs
 - Checking job efficiency
- Knowing when and how to ask for help

From "how do I submit?" to "am I using the right resources?"

How Your Job Moves Through Quest

Your laptop / Quest OnDemand



Quest login node



Slurm scheduler



CIERA compute resources

Key idea

- Login nodes are for preparing work
- Compute nodes are where jobs run
- Slurm decides when and where
 - Your request controls what Slurm looks for

Prepare on login nodes. Run on compute nodes.

CIERA Allocations on Quest

Allocation / account = which group/project your job uses

Partition = which hardware pool your job runs on

Resources = how much CPU, memory, GPU, and time you request

b1094 — CIERA

most common

General CIERA astronomy
research

ciera-std · ciera-gpu ·
ciera-himem

b1095 — Grail

Gravitational-wave
research

grail-std · grail-ligo

b1119 —

Trident/POSYDON

Stellar population
synthesis

posydon-std · posydon-
priority · posydon-long

Quick checks

```
groups # your allocation/group memberships  
sinfo -s # visible Slurm partitions
```

A job still needs a valid **--account** + **--partition** pair.

CIERA Cluster (b1094)

General CIERA astronomy research

Standard CPU — ciera-std

- 32 CPU nodes
- 52 cores per node
- 192 GB RAM per node
- InfiniBand HDR interconnect

Specialty — ciera-gpu / ciera-himem

- GPU: 1× NVIDIA A100 (40 GB)
- GPU: 2× NVIDIA L40S (48 GB each)
- High-memory nodes: 1 TB & 2 TB RAM
- 200 TB shared high-speed storage

Bonus: apply for **5 TB personal scratch** on Quest.

Grail and Trident at a Glance

Grail Cluster (b1095)

Gravitational-wave research

- NSF-funded cluster
- 18 general compute nodes
 - 128 cores per node
 - 512 GB RAM per node
- A30 and L40S GPU nodes

Trident Cluster (b1119)

Stellar population synthesis

- Supported by the Moore Foundation
 - 26 CPU compute nodes
 - Mix of 52- and 128-core nodes
- 192 GB or 512 GB RAM per node
- POSYDON-related partitions

Not sure which fits? Just ask.

What Kind of Job Do You Have?

Simple script

Start with **ciera-std**
Often 1 CPU is enough at first

Many independent runs

Use a **job array**
Set a concurrency limit

GPU code

Use **ciera-gpu**
Only if the code actually uses
GPUs

Large memory job

Use **ciera-himem**
Only when standard memory
isn't enough

Interactive debugging

Use a **short interactive job**
Good for testing, not
production

Request what your code can actually use, not what sounds powerful.

Interactive Jobs vs Batch Jobs

Interactive jobs

Work on a compute node live

- Testing, debugging, imports, file paths
 - Short, exploratory work
- Analysis needing more CPU/RAM than a laptop

Batch jobs

Submit and walk away

- Long runs and parameter sweeps
- Reproducible work you'll rerun later
 - Keeps running after you log out

```
srun -A b1094 -p ciera-std -t 00:20:00 --mem=4G --pty bash
```

Start interactive to explore, then move real runs into a batch script.

When Your Laptop Is Not Enough

Signs you've outgrown your laptop

- Needs more memory than you have
- Needs more CPUs to finish in time
 - Datasets too large to fit or move
- Runtime measured in hours or days

What to use instead

- Quick interactive work: an interactive job or Quest Analytics node
- Long or reproducible work: a batch job
 - Browser access to files and apps: Quest OnDemand

Explore interactively, but turn long production runs into batch jobs.

Quest OnDemand and Analytics Nodes

Quest OnDemand

Browser-based access, no SSH setup

- Files and a terminal
- Jupyter and RStudio
- Launch Jupyter, RStudio, MATLAB, etc.

Analytics Nodes

Browser-based interactive apps

- Jupyter, RStudio, SAS Studio
 - Same Quest files/storage
- Low/moderate interactive work

Use these for interactive work. Use batch jobs for long or heavy production runs.

Storage: Home, Projects, Scratch

/home (80GB)

- Code
- Small files
- Config files
- Personal scripts
- Limited space

/projects/<alloc>

- Shared research data
 - Group software
- Long-term project files
 - e.g. /projects/b1094

/scratch/<netid>

- Large temporary files
 - Job working dirs
- Fast temporary data
 - **Not backed up**
- Purged after inactivity

```
homedu # home usage
```

```
checkproject b1094 # project usage
```

```
du -sh * | sort -h # what's taking space here
```

Anatomy of a Slurm Job: Resource Request

```
#!/bin/bash

#SBATCH --job-name=my-job

#SBATCH --account=b1094

#SBATCH --partition=ciera-std

#SBATCH --time=00:30:00

#SBATCH --ntasks=1

#SBATCH --cpus-per-task=1

#SBATCH --mem=4G

#SBATCH --output=logs/%x-%j.out

#SBATCH --error=logs/%x-%j.err
```

The #SBATCH lines tell Slurm what the job needs.

Anatomy of a Slurm Job: Run Commands

```
module purge  
module load python/3.12.10  
  
cd "$SLURM_SUBMIT_DIR"  
python analysis.py
```

These run once the job starts on the compute node.

Reading a Slurm Script

Identity: --job-name

Where it runs: --account, --
partition

What it requests: --time, --ntasks, -
-cpus-per-task, --mem

Where logs go: --output, --error

Setup & work: module purge / load,
then your command

Choosing Resources Responsibly

Before submitting, ask:

- How long will this probably run?
- How much memory does it need?
- Serial, threaded, MPI, or GPU?
- One job or many similar jobs?
- A test, or a production run?

Slurm options

- Runtime: `--time`
- Memory: `--mem` or `--mem-per-cpu`
 - Processes: `--ntasks`
 - Threads: `--cpus-per-task`
 - GPUs: `--gres=gpu:...`
 - Allocation: `--account`
 - Partition: `--partition`

Start small, measure, then scale up.

A Healthier Batch Script

```
#!/bin/bash

#SBATCH --job-name=my-analysis

#SBATCH --account=b1094

#SBATCH --partition=ciera-std

#SBATCH --time=01:00:00

#SBATCH --cpus-per-task=1

#SBATCH --mem=4G

#SBATCH --output=logs/%x-%j.out
#SBATCH --error=logs/%x-%j.err


set -euo pipefail

module purge

module load python/3.12.10


cd "$SLURM_SUBMIT_DIR"

mkdir -p logs


echo "Job `SLURM_JOB_ID` on $(hostname) - started $(date)"

python analysis.py

echo "Finished $(date)"
```

Why It's Healthier

- **Clear job name** — easy to spot in the queue
- **Separate out/err logs** carry the job name & ID
- **Clean module environment** with module purge
- **Stops on common errors** via `set -euo pipefail`
- **Records job ID, node, and timestamps**
- **Easier to debug** later

The CIERA Batch-Script Generator

<https://miccles.github.io/batch-webpage/>

The screenshot shows the Northwestern CIERA Quest Job Builder web interface. At the top, it says "Northwestern CIERA" and "Quest Job Builder". Below this, it says "Choose an allocation and node type, describe your job, and request resources. The builder checks your request against the node's capacity and writes a Slurm script or interactive command you can copy." There are two tabs: "Batch job" (selected) and "Interactive session". The "Batch job" tab has a sub-tab "Generate an sbatch script". The main form is divided into several sections: "Allocation, node type & partition", "Job & resources", and "Per-node fit". The "Allocation, node type & partition" section has dropdowns for "Allocation" (CIERA (i3094)), "Node type" (Standard CPU nodes), and "Partition" (Ciera-std). The "Job & resources" section has input fields for "Job name" (my-simulation), "Execution model" (Serial), "Total memory per node (GB)" (4), and "Walltime" (01:00:00). The "Per-node fit" section shows "CPUs (serial)" (1 of 32 max) and "Memory per node" (4 GB of 192 GB max). A green box indicates "Request fits the selected node type." Below this, there is a "Copy" button and a "Download" button. The bottom right of the form shows a preview of the generated Slurm script, including comments about the job name, partition, and resource type, and a list of modules to load.

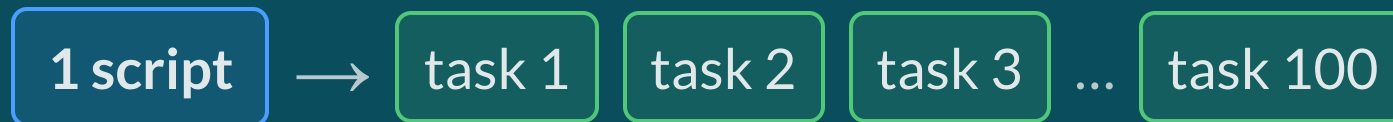
How to use it

1. Choose allocation
2. Choose partition
3. Select workload type
4. Enter time, CPUs, memory, GPUs
5. Generate the script
6. Read the script before submitting

The generator helps you **start correctly**. Still, read what the script requests.

Job Arrays – One Script, Many Runs

Once you can write a batch script, run the same one many times with different inputs.



Great for **100 stars • 50 simulations • 200 images • 500 parameter sets.**

Job Array Syntax

```
#SBATCH --array=1-100%10
```

- 1 - 100: task IDs 1 to 100
- %10: at most 10 run at once
- \$SLURM_ARRAY_TASK_ID: the current task number
- %A and %a: useful in output filenames

Arrays are cleaner than 100 nearly-identical scripts.

Job Array with an Input List

inputs.txt — a manifest, one input per line:

```
input01.dat  
input02.dat  
input03.dat  
input04.dat  
input05.dat
```

The Array Script

```
#!/bin/bash

#SBATCH --job-name=array-demo

#SBATCH --account=b1094

#SBATCH --partition=ciera-std

#SBATCH --time=00:30:00

#SBATCH --mem=4G

#SBATCH --array=1-5%2

#SBATCH --output=logs/%A_%a.out

#SBATCH --error=logs/%A_%a.err


module purge

module load python/3.12.10


INPUT=$(sed -n "${SLURM_ARRAY_TASK_ID}p" inputs.txt)

echo "Processing $INPUT"

python analyze.py "$INPUT"
```

Each task reads one line from `inputs.txt`. If task 3 fails, you know which input it used.

Large Arrays Need Concurrency Limits

Risky

```
#SBATCH --array=1-10000
```

All tasks may become eligible at once.

Better

```
#SBATCH --array=1-10000%20
```

At most 20 tasks run at the same time.

Less scheduler pressure

Easier to catch failures

Fairer cluster sharing

More simultaneous tasks does not always mean faster science.

What Is My Job Doing?

```
squeue -u $USER # your running + pending jobs  
squeue -j <jobid> # check one job  
scontrol show job <jobid> # detailed job info
```

R — running

PD — pending

CG — completing

F — failed

TIMEOUT

OUT_OF_MEMORY

Pending isn't an error. It often just means resources are busy.

Did My Job Use What I Requested?



```
seff <jobid>
```

```
sacct -j <jobid> --format=JobID,State,Elapsed,TotalCPU,ReqMem,MaxRSS,ExitCode
```

Checking Job Efficiency with seff

```
seff <jobid>
```

```
[quest_demo@quser23]$ seff 2261088
```

```
-----  
JOB EFFICIENCY  
-----
```

```
Job ID: 2261088  
Cluster: quest  
User/Group: netid/netid  
State: COMPLETED (exit code 0)  
Cores: 4  
CPU Utilized: 00:32:42  
CPU Efficiency: 24.84% of 00:32:42 core-walltime  
Job Wall-clock time: 00:32:42  
Memory Utilized: 366.84 MB  
Memory Efficiency: 3.66% of 10.00 GB
```

What it reports

- CPU efficiency: how much of the CPUs you used
- Memory efficiency: peak vs requested
- Wall-clock vs the time you asked for

Example seff output: what looks inefficient here?

Reading the Results

Low CPU efficiency

Requested more CPUs than
the code used

Low memory efficiency

Requested far more
memory than needed

OUT_OF_MEMORY

Memory was too low, or the
code used too much

TIMEOUT

Walltime was too short

Good efficiency

Reasonable request

Measure one job, then improve the next one.

When Jobs Fail

Exits immediately

Check .err, paths, typos

ModuleNotFoundError

Check your Python environment

command not found

Check modules & PATH

Out of memory

Check MaxRSS, request more memory

Timeout

Check elapsed time vs
walltime

Save the job ID, script, .out, and .err before asking for help.

Software Environments & Reproducibility

Software environments

- Modules for system software
- **conda** or **venv** for project packages
- Don't install everything into **base**
 - One environment per project
- Check **which python** you're using

Reproducibility habits

- Keep your batch script
- Save input lists & parameters
 - Save logs
- Record modules & environments
- Use Git; record random seeds

```
# clean environment
module purge
module load python-miniconda3
conda activate myenv
which python

# capture the run
module list 2> modules.txt
conda env export --from-history > environment.yml
git rev-parse HEAD > git-commit.txt
```

The batch script is part of the scientific record.

Healthy Habits on a Shared Cluster

- Don't run heavy work on login nodes
 - Start with small tests
- Request resources based on measurements
 - Use the correct partition
- Use concurrency limits for large arrays
 - Clean up temporary files
 - Cancel jobs you no longer need
- Don't repeatedly resubmit failing large jobs
- Ask before scaling to very large workloads

Good habits help everyone use the cluster more smoothly.

How to Ask for Help

Less helpful

"My job does not work."

Much more helpful

Job ID • allocation • partition

Script (attached / pasted)

Expected vs observed behavior

Error message

What you already tried

CIERA computing help

Michail Damoulakis — CIERA Computational Specialist

michail.damoulakis@northwestern.edu • ciera-computing@northwestern.edu

General Quest / RCS issues: quest-help@northwestern.edu

Before and After Every Job

Before submitting

- Using a compute node for real work?
 - Allocation correct?
 - Partition appropriate?
- Code uses the CPUs/GPUs requested?
 - Time & memory reasonable?
 - Output/error files identifiable?
 - Large array concurrency-limited?
 - Temporary data going to scratch?

After completion

- Did the job finish successfully?
 - What did seff report?
 - Maximum memory used?
 - Too many CPUs requested?
 - Saved the script and logs?
- Can I improve the next submission?

Start small → measure → adjust → scale up

Questions?

You do not need to remember everything today.

Three takeaways

1. Use the **right CIERA resource** for the job
2. Write **readable, reproducible** batch scripts
3. **Ask for help early**, with the job ID and logs

Michail Damoulakis • CIERA Computational Specialist

michail.damoulakis@northwestern.edu • ciera-computing@northwestern.edu

Appendix – Reference Material

Not part of the main talk

Appendix A – Useful Commands

```
# Check groups / allocations

groups

id $USER


# Check partitions

sinfo -s

sinfo -p ciera-std

sinfo -p ciera-gpu -N -o "%N %C %m %G %T"


# Submit and monitor

sbatch job.sh

squeue -u $USER

sacct -j <jobid>

seff <jobid>


# Cancel a job

scancel <jobid>
```

Appendix B – CPU, GPU, and High-Memory Jobs

CPU jobs

- Most scripts start here
- Serial, threaded, or many independent tasks
- Use **ciera-std** for general CIERA jobs

GPU jobs

- Only if the code supports GPU acceleration
 - Common in ML, some simulations, CUDA tools
- Request GPUs explicitly

High-memory jobs

- When standard nodes lack RAM
 - Good for large in-memory datasets
- Limited resource — ask if unsure

